

Earshot ML & Backend Code Review

Synchronized review · alexanderli07/Earshot · main @ 8fac3c9

RELEASE RECOMMENDATION

NO-GO

Hold unattended and safety-adjacent release until the minimum blockers on page 8 are closed.

Executive summary

The offline ML core is materially hardened, but the deployed backend exposes an unauthenticated control plane and cannot guarantee alarm priority, truthful delivery status, supervised microphone liveness, bounded/private teaching, or its configured visual cues. These failures can suppress or misreport safety-adjacent alerts even while health appears green.

6**CRITICAL****6****HIGH****3****MODERATE**

MVP context

Offline Raspberry Pi

Local 16 kHz YAMNet inference after the one-time pinned artifact download.

Under two seconds

An acceptance target to measure on the actual Pi, microphone, room, and callback load; not a proven bound.

Three-clip teaching

The product contract says three clips. Current backend and engine accept any non-empty count.

Safety and accessibility

Not certified and must not replace approved alarms. Visual and vibration semantics still need dependable implementation.

BOTTOM LINE

The strongest code is in ML contract and persistence handling; the release risk is concentrated at the backend control, lifecycle, output arbitration, and delivery boundaries.

EVIDENCE

Synchronization and validation status

REMOTE	origin = github.com/alexanderli07/Earshot.git	CONFIRMED
SNAPSHOT	branch main · commit 8fac3c9df26e603f9fea34754a07b5abc31ab9e8	PINNED
LINE BASIS	All cited files reviewed against the same local commit.	ALIGNED
PRE-PDF TREE	git status --short --untracked-files=all returned empty.	CLEAN
NETWORK	No fetch or external request; synchronization is commit-pinned.	LOCAL ONLY

Fresh automated validation

ML fast suite

176 passed · 1 integration test deselected · 0 failures. Ran locally with bytecode and pytest cache disabled.

Backend suite

8 passed · 0 failures. One third-party TestClient deprecation warning; no network or hardware used.

NOT EXERCISED

Production model integration, target Pi/microphone, GPIO patterns, ntfy delivery, and end-to-end latency were not exercised. The under-two-second goal remains unvalidated hardware acceptance.

ML assessment

No critical tensor/window/WAV/model/store consistency defect found.

CONTRACTS	Finite 15,600-sample waveform input; one float32 521-score output, one float32 1,024-embedding output, and exactly 521 class-map rows.
ARTIFACTS	Pinned SHA-256 digests, bounded/timed streaming, process-unique partials, verification before atomic replacement.
CAPTURE	LatestBlockQueue is bounded at two blocks and drops oldest data under load; stream closure uses a finally path. A gap-to-detector reset is still a High finding.
STATE	YAMNet inference and TeachStore use locks; store loading validates shape/value contracts; mutations roll back and NPZ persistence uses atomic replacement.

Assessment boundary: robust code contracts are not evidence of acoustic accuracy, alert effectiveness, or safety certification.

RELEASE BLOCKING

Critical findings 1-2

Scenario states the concrete failure mode; fix states the minimum engineering direction.

CRITICAL 01

Unauthenticated network control plane

backend/app/config.py:24; backend/app/main.py:67-68,79-161; backend/app/core.py:105

SCENARIO

The service binds to every interface, permits wildcard CORS, and authenticates neither REST nor WebSocket traffic. Any hotspot or LAN peer can inject debug alerts, read event history, upload teaching audio, and persistently mute or reprioritize labels.

FIX

Default to a constrained interface; require authenticated, authorized roles on data and mutation paths including WebSocket; narrow CORS; disable debug in production; add request limits and rate controls.

CRITICAL 02

Low-priority alerts can cancel smoke alarm; stale worker motor race

backend/app/sinks.py:73-99

SCENARIO

Every new event increments one generation counter regardless of urgency, so a low alert supersedes an active high smoke/fire alert. A stale worker then exits its loop and unconditionally writes motor=0, which can switch off the newer worker after it has started.

FIX

Use one serialized hardware controller with explicit priority, hold/latch, and coalescing rules. Only the current owner may write or clear outputs; guard every transition and add deterministic preemption/race tests.

RELEASE BLOCKING

Critical findings 3-4

Scenario states the concrete failure mode; fix states the minimum engineering direction.

CRITICAL 03

Configured strobe, pulse, and blink cues are never implemented

backend/app/config.py:39-44; backend/app/sinks.py:82-99

SCENARIO

Profiles advertise strobe, pulse, and blink, but Alerts only assigns a solid RGB color and runs the motor timing list. The named LED pattern is merely printed in mock mode. Demo and accessibility behavior therefore diverge from configuration and documentation.

FIX

Implement timed LED patterns in the serialized alert scheduler, including cancellation and final-state rules, or remove the unsupported profile claims. Verify transitions with a fake clock and GPIO adapter.

CRITICAL 04

False-green ML health and unmanaged thread/microphone lifecycle

backend/app/ml_bridge.py:23-57; backend/app/main.py:48-60, 98-99; ml/earshot_ml/pipeline.py:179, 208-209

SCENARIO

ML health reflects import/initial construction, not listener liveness. A startup device fault or later disconnect can kill the daemon worker while /healthz remains green. Shutdown closes alerts and HTTP but never signals or joins ML, so stream closure is not coordinated with app lifecycle.

FIX

Add a stop Event, startup handshake, supervised worker, captured last error/frame time, and alive-state health. On shutdown: signal, join with a deadline, then report failure. Define a bounded restart/degraded-mode policy.

RELEASE BLOCKING

Critical findings 5-6

Scenario states the concrete failure mode; fix states the minimum engineering direction.

CRITICAL 05

Unbounded blocking /teach retains raw audio

backend/app/main.py:131-136; backend/app/ml_bridge.py:59-75; ml/earshot_ml/core.py:393-415

SCENARIO

The async route reads every upload fully into memory, accepts unbounded count/size, performs blocking decode/inference on the event loop, and writes raw clips to mkdtemp directories that are never removed. Repetition can exhaust RAM/disk, stall APIs, and retain private voice audio.

FIX

Require exactly three clips; cap request bytes, clip bytes, decoded duration, and concurrency; stream into TemporaryDirectory; clean up in finally; validate WAV early; run teaching in a bounded worker with timeout and observable failure.

CRITICAL 06

Delivery reports success even when every sink fails

backend/app/core.py:154-166; backend/app/main.py:98,117-127; backend/app/sinks.py:39-43,73-99,121-131

SCENARIO

Dispatcher records the event, gathers with return_exceptions, and discards outcomes. WebSocket and ntfy swallow errors; GPIO returns after spawning a thread. /debug/event sets delivered=true whenever the event is not muted, even if no client, phone, LED, or motor received it.

FIX

Separate accepted from delivered. Make sink adapters return structured outcomes or raise; apply required-sink policy, deadlines, retries, and telemetry; expose per-sink health and report failure when all configured delivery paths fail.

RELEASE HARDENING

High findings

These issues materially affect correctness, availability, or confidence and should close before MVP release.

HIGH 01**Dropped capture gaps do not reset detector streaks**

ml/earshot_ml/pipeline.py:173-221; ml/earshot_ml/core.py:67-96

SCENARIO

A queue sequence gap clears the audio buffer, but EventDetector is not told. Two qualifying windows separated by dropped audio may still satisfy a consecutive-window gate.

FIX

Propagate a discontinuity marker and reset streaks before the next window; test dropped blocks between positives.

HIGH 02**Optional ML errors can stop backend startup**

backend/app/main.py:35-51; backend/app/ml_bridge.py:23-51

SCENARIO

start() degrades only on FileNotFoundError. Corrupt model/class map/store, interpreter, or other construction failures escape lifespan and can prevent the supposedly optional backend from starting.

FIX

Catch documented ML domain failures into an explicit degraded state and expose the reason; make fail-open versus fail-closed configurable.

HIGH 03**Rules writes are nontransactional in memory**

backend/app/core.py:93-119

SCENARIO

Rules.set mutates _rules before persistence. If mkdir, write, or replace fails, the request errors but live memory keeps the unsaved rule, diverging from disk and caller expectations.

FIX

Use copy-on-write: validate and persist a candidate, then swap live state; otherwise roll back under a lock.

HIGH 04**Fan-out permits thread and task exhaustion**

backend/app/core.py:154-166; backend/app/sinks.py:73-78

SCENARIO

Each accepted event schedules executor work and Alerts creates a daemon thread. With no bounded queue, coalescing, or rate limit, debug/API floods can exhaust workers and make alert timing nondeterministic.

FIX

Use bounded queues and fixed workers, reject or coalesce overflow by policy, and rate-limit authenticated producers.

HIGH 05**Exactly-three-clips contract is not enforced**

backend/app/main.py:131-136; ml/earshot_ml/core.py:372-395

SCENARIO

The endpoint and MVP text say three clips, while the route and engine accept any non-empty iterable. One clip weakens teaching quality; large counts amplify the critical resource issue.

FIX

Enforce len(clips)==3 at API and engine boundaries, return a clear 422, and test 0/1/2/3/4 counts.

HIGH 06**Critical backend behaviors lack tests**

backend/tests/test_backend.py:1-129

SCENARIO

Eight tests cover core happy paths and one debug flow, but not authorization, all-sink failure, priority preemption/races, worker death, teach limits/cleanup, rule-save rollback, or shutdown.

FIX

Add endpoint, fault-injection, lifecycle, and deterministic concurrency tests; retain separate target-hardware acceptance.

CORRECTNESS & CLARITY

Moderate findings

MODERATE 01

Non-finite event values pass normalization

backend/app/core.py:25-49

SCENARIO

float("nan") and infinities survive confidence/timestamp conversion and can poison JSON responses or clients; confidence is not range-checked.

FIX

Require finite values, clamp or reject confidence outside 0..1, and validate timestamp policy before storage.

MODERATE 02

API errors use inconsistent HTTP status semantics

backend/app/main.py:131-161

SCENARIO

Teach RuntimeError and invalid rule urgency return HTTP 200 with an error object, while framework validation returns 422. Clients cannot rely on status for retries or user feedback.

FIX

Use HTTPException and one error envelope: 4xx for bad input/auth, 409 for conflicts, and 503 for unavailable ML/sinks.

MODERATE 03

Backend README carries a stale mode-gate warning

backend/README.md:66-71; backend/app/ml_bridge.py:59-75; ml/earshot_ml/core.py:372-415

SCENARIO

The README says live teaching is blocked by an ML mode gate, but current ML serializes interpreter access and has no such gate. The warning hides the real blocking/temp-file risks.

FIX

Remove the stale claim and document current synchronous teaching, limits, cleanup, and lifecycle behavior.

ML review conclusion

POSITIVE, WITH ONE INTEGRATION CAVEAT

No critical tensor/window/WAV/model/store consistency defect was found in the reviewed ML implementation.

- Model contract checks reject wrong input/output shape, dtype, non-finite values, and invalid class-map cardinality.
- WAV handling validates supported PCM, mono conversion, resampling, window geometry, and malformed input boundaries.
- Artifacts are checksummed and size-bounded before atomic installation; existing known-good destinations are protected.
- Capture buffering is bounded; inference and taught-state access are locked; store load/save and teach mutations preserve consistency.

Caveat: dropped capture gaps clear the audio buffer but do not reset detector streak state (High 01).

EXIT CRITERIA

Minimum release-blocking priorities

This is the smallest defensible closure set for an unattended, safety-adjacent MVP. It does not confer safety certification.

01

Secure the control plane

Authenticate and authorize REST/WebSocket access; narrow CORS and bind exposure; disable production debug; add request/rate limits.

02

Make alert output deterministic

Replace per-event threads with one priority-aware controller; latch high alerts; fix stale writes; implement and test strobe/pulse/blink.

03

Supervise ML and reset on capture gaps

Own stop/join/restart, truthful liveness and last-error health, deterministic microphone closure, and detector reset after dropped audio.

04

Bound and privatize teaching

Require exactly three validated clips, enforce byte/duration/concurrency limits, offload work, time out, and delete raw temporary audio in finally.

05

Report delivery truthfully

Return accepted versus per-sink delivered outcomes, fail when all configured sinks fail, and add deadlines, retries, and delivery telemetry.

06

Prove faults and target hardware

Add missing auth/failure/concurrency/lifecycle tests, then run model, Pi microphone, GPIO, ntfy, alarm-priority, and measured under-two-second acceptance.

RELEASE GATE

Release only after all six priorities pass automated fault tests and a documented target-Pi run. Until then: no-go for unattended or safety-adjacent deployment.